In [2]:
```
# imports
%pip install seaborn -qq
%pip install grad-cam -qq

import medmnist
import os
import torch
import numpy as np
import torch.nn as nn
import seaborn as sns
import matplotlib.pyplot as plt
from medmnist import BloodMNIST
import torchvision.models as models
from torch.utils.data import DataLoader
import torchvision.transforms as transforms
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget

# check if we have CUDA acceleration
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"using: {device}")
```

```
Note: you may need to restart the kernel to use updated packages.
Note: you may need to restart the kernel to use updated packages.
using: cuda
```

Transforms

In [3]:
```
"""
Image transformation pipelines for model training and evaluation.
- train_transforms: Applies random augmentations (flips, rotation, color jitter) to
- test_transforms: Standardizes validation/test data (tensor conversion, ImageNet n
"""

# image data augmentation for training
train_transforms = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(25),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # I
])

# no augmentation for val or test
test_transforms = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # I
])
```

Load Datasets

In [32]:
```
"""
Directory configuration and dataset initialization.
- Sets file paths for the BloodMNIST data and output figures.
- Loads the train, validation, and test splits, applying the appropriate transforma
"""

# change the data_folder path if needed
data_folder = "/blue/bme6938/gael.garcia/bme6938-project-2/data"
figures_folder = "/blue/bme6938/gael.garcia/bme6938-project-2/figures"
models_folder = "/blue/bme6938/gael.garcia/bme6938-project-2/models"

# reload the data with the transforms applied
train_dataset = BloodMNIST(split='train', download=False, size=64, root=data_folder
val_dataset   = BloodMNIST(split='val',   download=False, size=64, root=data_folder
test_dataset  = BloodMNIST(split='test',  download=False, size=64, root=data_folder

print(f"Training dataset length: {len(train_dataset)}")
print(f"Training dataset length: {len(val_dataset)}")
print(f"Training dataset length: {len(test_dataset)}")
```

```
Training dataset length: 11959
Training dataset length: 1712
Training dataset length: 3421
```

DataLoaders

In [8]:
```
"""
Data iterators to handle batching during the training loop.
- train_loader: Batches the data (size 64) and shuffles to ensure random sampling d
- val_loader & test_loader: Batches the data sequentially (no shuffling) for consis
"""

# define our dataloaders
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False) # note that we D
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

Model 1: Custom CNN (from scratch)

In [9]:
```
"""
Custom Convolutional Neural Network (CNN) architecture.
- Features: 4 sequential convolutional blocks (Conv2d, BatchNorm, ReLU, MaxPool) to
- Classifier: Uses global average pooling, dense layers, and dropout to map the ext
"""

# create a custom CNN from scratch
class CustomCNN(nn.Module):
```

```
    def __init__(self, num_classes=8):
        super(CustomCNN, self).__init__()

        # extract image features
        # sequential allows us to define multiple layers sequentially (one after th
        self.features = nn.Sequential(

            # block 1
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),                          # activation function
            nn.MaxPool2d(2, 2),

            # block 2
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),                          # activation function
            nn.MaxPool2d(2, 2),

            # block 3
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),                          # activation function
            nn.MaxPool2d(2, 2),

            # block 4
            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(),                          # activation function
            nn.MaxPool2d(2, 2),

        )

        # preform classification
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((1,1)),
            nn.Flatten(),                       # convert to 1D vector
            nn.Linear(256, 128),
            nn.ReLU(),                          # activation function
            nn.Dropout(0.5),                    # dropout helps prevent overfitting
            nn.Linear(128, num_classes),        # want the output to be limited to n
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

model = CustomCNN().to(device)
print(model)
```

```
CustomCNN(
  (features): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=T
rue)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=T
rue)
    (6): ReLU()
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=
True)
    (10): ReLU()
    (11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (12): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (13): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats
=True)
    (14): ReLU()
    (15): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): AdaptiveAvgPool2d(output_size=(1, 1))
    (1): Flatten(start_dim=1, end_dim=-1)
    (2): Linear(in_features=256, out_features=128, bias=True)
    (3): ReLU()
    (4): Dropout(p=0.5, inplace=False)
    (5): Linear(in_features=128, out_features=8, bias=True)
  )
)
```

Sanity Check

In [10]:

```
"""
Sanity Check: Pass a dummy batch through the model to verify output dimensions
Input: [4, 3, 64, 64] -> (Batch Size, RGB Channels, Height, Width)
Expected Output: [4, 8] -> (Batch Size, Number of Classes)
"""

with torch.no_grad():
    dummy = torch.rand(4, 3, 64, 64).to(device)
    out = model(dummy)
    print(f"Shape: {out.shape}")
```

```
Shape: torch.Size([4, 8])
```

Loss + Optimizer

In [11]:

```
"""
Training configuration setup.
- criterion: Uses Cross-Entropy to calculate the error for our multi-class classifi
```

```
- optimizer: Uses Adam to update the model's weights based on the computed gradient
- scheduler: Dynamically drops the learning rate if the validation loss plateaus to
"""

criterion = nn.CrossEntropyLoss() # this is our loss function that penalizes the mo
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3) # Adam is the adpative op

# learning rate scheduler: reduces the LR when validation loss stops improving.
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', patie
```

Training Loop

In [12]:

```
"""
Complete training loop for the PyTorch model.
- Handles both training (gradient updates) and validation (evaluation) phases per e
- Implements early stopping to prevent overfitting and saves the best model weights
- Tracks and returns loss and accuracy metrics for both phases across all epochs.
"""

# create a function used for training a provided model
def train_model(model, train_loader, val_loader, model_name, num_epochs=50, patienc
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', p

    best_val_loss = float('inf')
    epochs_no_improve = 0
    train_losses, val_losses, train_accs, val_accs = [], [], [], [] # track metrics

    for epoch in range(num_epochs):
        # --- training ---
        model.train()                                       # set model to training
        running_loss, correct, total = 0, 0, 0
        for images, labels in train_loader:
            images = images.to(device)
            labels = labels.squeeze().long().to(device)

            optimizer.zero_grad()                           # clear old gradients f
            outputs = model(images)                         # forward pass
            loss = criterion(outputs, labels)               # compute error
            loss.backward()                                 # compute gradients (ba
            optimizer.step()                                # update model weights

            running_loss += loss.item()
            _, predicted = outputs.max(1)                   # get the index of the
            correct += predicted.eq(labels).sum().item()
            total += labels.size(0)

        train_loss = running_loss / len(train_loader)
        train_acc  = correct / total

        # --- validation ---
        model.eval()                                        # set model to evaluati
        val_loss, correct, total = 0, 0, 0
```

```
        with torch.no_grad():                                # disable gradient calc
            for images, labels in val_loader:
                images = images.to(device)
                labels = labels.squeeze().long().to(device)

                outputs = model(images)
                loss = criterion(outputs, labels)

                val_loss += loss.item()
                _, predicted = outputs.max(1)
                correct += predicted.eq(labels).sum().item()
                total += labels.size(0)

        val_loss = val_loss / len(val_loader)
        val_acc  = correct / total

        train_losses.append(train_loss)
        val_losses.append(val_loss)
        train_accs.append(train_acc)
        val_accs.append(val_acc)
        scheduler.step(val_loss)                             # adjust learning rate

        # --- early stopping + save best ---
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            epochs_no_improve = 0
            torch.save(model.state_dict(), os.path.join(models_folder, f"best_{mode
        else:
            epochs_no_improve += 1
            if epochs_no_improve >= patience:                        # stop early i
                print(f"early stopping triggered at epoch {epoch+1}")
                break

        print(f"[{model_name}] epoch {epoch+1:02d}/{num_epochs} | train loss: {trai

    print(f"{model_name} done!")
    return train_losses, val_losses, train_accs, val_accs
```

In [13]:

```
print("=== training scratch CNN ===")

# training for max 50 epochs,, 50 was defined in the function, no need to include i
cnn_results = train_model(model, train_loader, val_loader, "cnn")
```

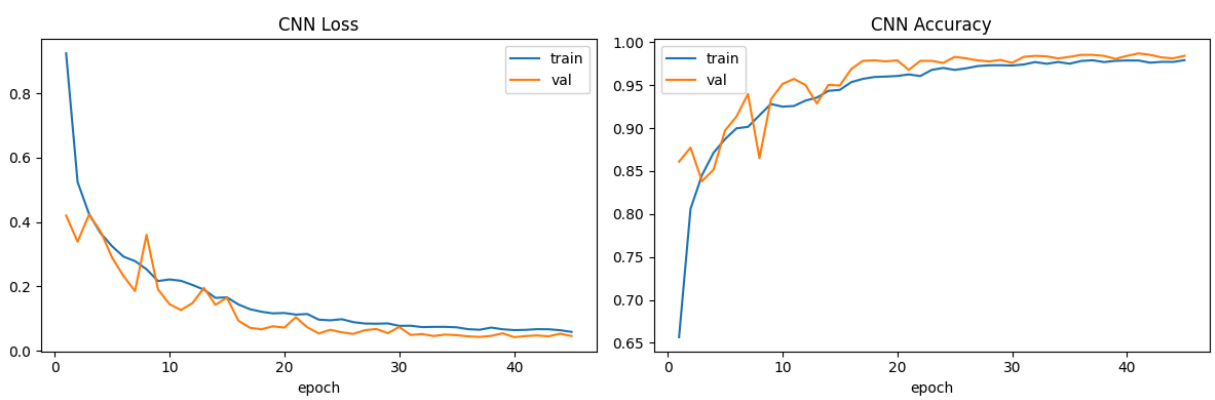
```
=== training scratch CNN ===
[cnn] epoch 01/50 | train loss: 0.9239 acc: 0.6566 | val loss: 0.4205 acc: 0.8610
[cnn] epoch 02/50 | train loss: 0.5243 acc: 0.8060 | val loss: 0.3390 acc: 0.8773
[cnn] epoch 03/50 | train loss: 0.4251 acc: 0.8454 | val loss: 0.4234 acc: 0.8382
[cnn] epoch 04/50 | train loss: 0.3657 acc: 0.8713 | val loss: 0.3711 acc: 0.8516
[cnn] epoch 05/50 | train loss: 0.3251 acc: 0.8872 | val loss: 0.2902 acc: 0.8972
[cnn] epoch 06/50 | train loss: 0.2927 acc: 0.8998 | val loss: 0.2324 acc: 0.9136
[cnn] epoch 07/50 | train loss: 0.2789 acc: 0.9015 | val loss: 0.1859 acc: 0.9398
[cnn] epoch 08/50 | train loss: 0.2536 acc: 0.9150 | val loss: 0.3605 acc: 0.8651
[cnn] epoch 09/50 | train loss: 0.2170 acc: 0.9282 | val loss: 0.1912 acc: 0.9334
[cnn] epoch 10/50 | train loss: 0.2219 acc: 0.9251 | val loss: 0.1452 acc: 0.9515
[cnn] epoch 11/50 | train loss: 0.2179 acc: 0.9258 | val loss: 0.1269 acc: 0.9574
[cnn] epoch 12/50 | train loss: 0.2048 acc: 0.9321 | val loss: 0.1484 acc: 0.9504
[cnn] epoch 13/50 | train loss: 0.1903 acc: 0.9356 | val loss: 0.1956 acc: 0.9287
[cnn] epoch 14/50 | train loss: 0.1647 acc: 0.9435 | val loss: 0.1431 acc: 0.9504
[cnn] epoch 15/50 | train loss: 0.1664 acc: 0.9446 | val loss: 0.1654 acc: 0.9498
[cnn] epoch 16/50 | train loss: 0.1443 acc: 0.9537 | val loss: 0.0941 acc: 0.9690
[cnn] epoch 17/50 | train loss: 0.1296 acc: 0.9574 | val loss: 0.0718 acc: 0.9784
[cnn] epoch 18/50 | train loss: 0.1217 acc: 0.9596 | val loss: 0.0672 acc: 0.9790
[cnn] epoch 19/50 | train loss: 0.1166 acc: 0.9601 | val loss: 0.0764 acc: 0.9778
[cnn] epoch 20/50 | train loss: 0.1178 acc: 0.9608 | val loss: 0.0725 acc: 0.9790
[cnn] epoch 21/50 | train loss: 0.1127 acc: 0.9626 | val loss: 0.1048 acc: 0.9679
[cnn] epoch 22/50 | train loss: 0.1146 acc: 0.9607 | val loss: 0.0734 acc: 0.9784
[cnn] epoch 23/50 | train loss: 0.0971 acc: 0.9679 | val loss: 0.0543 acc: 0.9784
[cnn] epoch 24/50 | train loss: 0.0951 acc: 0.9701 | val loss: 0.0655 acc: 0.9761
[cnn] epoch 25/50 | train loss: 0.0984 acc: 0.9679 | val loss: 0.0578 acc: 0.9831
[cnn] epoch 26/50 | train loss: 0.0894 acc: 0.9697 | val loss: 0.0530 acc: 0.9813
[cnn] epoch 27/50 | train loss: 0.0852 acc: 0.9723 | val loss: 0.0642 acc: 0.9790
[cnn] epoch 28/50 | train loss: 0.0845 acc: 0.9732 | val loss: 0.0682 acc: 0.9778
[cnn] epoch 29/50 | train loss: 0.0855 acc: 0.9733 | val loss: 0.0553 acc: 0.9796
[cnn] epoch 30/50 | train loss: 0.0779 acc: 0.9730 | val loss: 0.0743 acc: 0.9761
[cnn] epoch 31/50 | train loss: 0.0783 acc: 0.9742 | val loss: 0.0496 acc: 0.9831
[cnn] epoch 32/50 | train loss: 0.0740 acc: 0.9770 | val loss: 0.0525 acc: 0.9842
[cnn] epoch 33/50 | train loss: 0.0747 acc: 0.9750 | val loss: 0.0463 acc: 0.9836
[cnn] epoch 34/50 | train loss: 0.0747 acc: 0.9771 | val loss: 0.0507 acc: 0.9813
[cnn] epoch 35/50 | train loss: 0.0733 acc: 0.9751 | val loss: 0.0490 acc: 0.9831
[cnn] epoch 36/50 | train loss: 0.0675 acc: 0.9783 | val loss: 0.0452 acc: 0.9854
[cnn] epoch 37/50 | train loss: 0.0658 acc: 0.9791 | val loss: 0.0435 acc: 0.9854
[cnn] epoch 38/50 | train loss: 0.0725 acc: 0.9770 | val loss: 0.0466 acc: 0.9842
[cnn] epoch 39/50 | train loss: 0.0672 acc: 0.9784 | val loss: 0.0548 acc: 0.9807
[cnn] epoch 40/50 | train loss: 0.0647 acc: 0.9788 | val loss: 0.0427 acc: 0.9842
[cnn] epoch 41/50 | train loss: 0.0654 acc: 0.9788 | val loss: 0.0461 acc: 0.9871
[cnn] epoch 42/50 | train loss: 0.0677 acc: 0.9763 | val loss: 0.0481 acc: 0.9854
[cnn] epoch 43/50 | train loss: 0.0673 acc: 0.9773 | val loss: 0.0454 acc: 0.9825
[cnn] epoch 44/50 | train loss: 0.0644 acc: 0.9772 | val loss: 0.0535 acc: 0.9813
early stopping triggered at epoch 45
cnn done!
```

Training Curve Plot

In [26]:

```
"""
Visualization of the training and validation metrics for the CNN trained from scrat
- Unpacks the results from the training loop.
- Generates a side-by-side plot of Loss and Accuracy across epochs.
```

```
- Saves the generated figure to the designated figures folder.
"""

# unpack the results returned from the train_model function
cnn_train_losses, cnn_val_losses, cnn_train_accs, cnn_val_accs = cnn_results

epochs_ran = range(1, len(cnn_train_losses) + 1)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# plot 1: loss
ax1.plot(epochs_ran, cnn_train_losses, label='train')
ax1.plot(epochs_ran, cnn_val_losses, label='val')
ax1.set_title('CNN Loss')
ax1.set_xlabel('epoch')
ax1.legend()

# plot 2: accuracy
ax2.plot(epochs_ran, cnn_train_accs, label='train')
ax2.plot(epochs_ran, cnn_val_accs, label='val')
ax2.set_title('CNN Accuracy')
ax2.set_xlabel('epoch')
ax2.legend()

plt.tight_layout()
plt.savefig('cnn_training_curves.png', dpi=150)
plt.show()

plt.savefig(os.path.join(figures_folder, 'cnn_training_curve.png'), dpi=150)
```




```
<Figure size 640x480 with 0 Axes>
```

Pretrained Models (ResNet and EfficientNet)

In [15]:

```
"""
Pre-trained model initialization for Transfer Learning.
- Loads ResNet50 and EfficientNet-B0 architectures with standard ImageNet weights.
- Replaces the final classification layers to output our specific 8 classes instead
"""

# load pretrained ResNet50 model
def get_resnet50(num_classes=8):
```

```
    model = models.resnet50(weights='IMAGENET1K_V1')

    # swap the final fully connected (fc) layer to output 8 classes
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model

# load pretrained EfficientNet-B0 model
def get_efficientnet(num_classes=8):
    model = models.efficientnet_b0(weights='IMAGENET1K_V1')

    # swap the final linear layer in the classifier block
    model.classifier[1] = nn.Linear(model.classifier[1].in_features, num_classes)
    return model

resnet_model = get_resnet50().to(device)
efficientnet_model = get_efficientnet().to(device)

print("resnet50 ready")
print("efficientnet_b0 ready")
```

```
resnet50 ready
efficientnet_b0 ready
```

In [16]:

```
print("\n=== training resnet50 ===")
# training for max 50 epochs,, 50 was defined in the function, no need to include i
resnet_results = train_model(resnet_model, train_loader, val_loader, "resnet50")

print("\n=== training efficientnet ===")
# training for max 50 epochs,, 50 was defined in the function, no need to include i
efficientnet_results = train_model(efficientnet_model, train_loader, val_loader, "e
```

```
=== training resnet50 ===
[resnet50] epoch 01/50 | train loss: 0.4668 acc: 0.8424 | val loss: 0.1659 acc: 0.93
69
[resnet50] epoch 02/50 | train loss: 0.2771 acc: 0.9070 | val loss: 0.1518 acc: 0.94
80
[resnet50] epoch 03/50 | train loss: 0.2107 acc: 0.9303 | val loss: 0.1642 acc: 0.94
51
[resnet50] epoch 04/50 | train loss: 0.1712 acc: 0.9405 | val loss: 0.1099 acc: 0.96
03
[resnet50] epoch 05/50 | train loss: 0.1536 acc: 0.9453 | val loss: 0.1194 acc: 0.95
62
[resnet50] epoch 06/50 | train loss: 0.1325 acc: 0.9538 | val loss: 0.1911 acc: 0.93
98
[resnet50] epoch 07/50 | train loss: 0.1294 acc: 0.9549 | val loss: 0.0767 acc: 0.97
25
[resnet50] epoch 08/50 | train loss: 0.1182 acc: 0.9599 | val loss: 0.1146 acc: 0.96
09
[resnet50] epoch 09/50 | train loss: 0.1101 acc: 0.9621 | val loss: 0.1319 acc: 0.95
44
[resnet50] epoch 10/50 | train loss: 0.1101 acc: 0.9620 | val loss: 0.1536 acc: 0.94
92
[resnet50] epoch 11/50 | train loss: 0.1108 acc: 0.9631 | val loss: 0.1481 acc: 0.95
44
early stopping triggered at epoch 12
resnet50 done!

=== training efficientnet ===
[efficientnet] epoch 01/50 | train loss: 0.4909 acc: 0.8369 | val loss: 0.2162 acc:
0.9235
[efficientnet] epoch 02/50 | train loss: 0.2109 acc: 0.9325 | val loss: 0.0824 acc:
0.9737
[efficientnet] epoch 03/50 | train loss: 0.1487 acc: 0.9516 | val loss: 0.0913 acc:
0.9679
[efficientnet] epoch 04/50 | train loss: 0.1202 acc: 0.9611 | val loss: 0.0836 acc:
0.9702
[efficientnet] epoch 05/50 | train loss: 0.1130 acc: 0.9645 | val loss: 0.0644 acc:
0.9784
[efficientnet] epoch 06/50 | train loss: 0.0974 acc: 0.9689 | val loss: 0.0886 acc:
0.9720
[efficientnet] epoch 07/50 | train loss: 0.0932 acc: 0.9698 | val loss: 0.0678 acc:
0.9766
[efficientnet] epoch 08/50 | train loss: 0.0881 acc: 0.9706 | val loss: 0.0520 acc:
0.9836
[efficientnet] epoch 09/50 | train loss: 0.0745 acc: 0.9754 | val loss: 0.0677 acc:
0.9772
[efficientnet] epoch 10/50 | train loss: 0.0750 acc: 0.9758 | val loss: 0.0373 acc:
0.9860
[efficientnet] epoch 11/50 | train loss: 0.0670 acc: 0.9763 | val loss: 0.0741 acc:
0.9743
[efficientnet] epoch 12/50 | train loss: 0.0708 acc: 0.9773 | val loss: 0.0383 acc:
0.9866
[efficientnet] epoch 13/50 | train loss: 0.0659 acc: 0.9779 | val loss: 0.0525 acc:
0.9842
[efficientnet] epoch 14/50 | train loss: 0.0676 acc: 0.9777 | val loss: 0.0487 acc:
0.9819
[efficientnet] epoch 15/50 | train loss: 0.0440 acc: 0.9855 | val loss: 0.0363 acc:
```

```
0.9877
[efficientnet] epoch 16/50 | train loss: 0.0359 acc: 0.9890 | val loss: 0.0349 acc:
0.9895
[efficientnet] epoch 17/50 | train loss: 0.0312 acc: 0.9900 | val loss: 0.0349 acc:
0.9901
[efficientnet] epoch 18/50 | train loss: 0.0335 acc: 0.9883 | val loss: 0.0425 acc:
0.9907
[efficientnet] epoch 19/50 | train loss: 0.0269 acc: 0.9905 | val loss: 0.0332 acc:
0.9924
[efficientnet] epoch 20/50 | train loss: 0.0360 acc: 0.9883 | val loss: 0.0388 acc:
0.9912
[efficientnet] epoch 21/50 | train loss: 0.0318 acc: 0.9873 | val loss: 0.0324 acc:
0.9901
[efficientnet] epoch 22/50 | train loss: 0.0357 acc: 0.9869 | val loss: 0.0560 acc:
0.9831
[efficientnet] epoch 23/50 | train loss: 0.0310 acc: 0.9884 | val loss: 0.0444 acc:
0.9871
[efficientnet] epoch 24/50 | train loss: 0.0308 acc: 0.9895 | val loss: 0.0379 acc:
0.9895
[efficientnet] epoch 25/50 | train loss: 0.0338 acc: 0.9885 | val loss: 0.0422 acc:
0.9842
[efficientnet] epoch 26/50 | train loss: 0.0219 acc: 0.9926 | val loss: 0.0320 acc:
0.9895
[efficientnet] epoch 27/50 | train loss: 0.0245 acc: 0.9919 | val loss: 0.0342 acc:
0.9895
[efficientnet] epoch 28/50 | train loss: 0.0208 acc: 0.9926 | val loss: 0.0310 acc:
0.9901
[efficientnet] epoch 29/50 | train loss: 0.0181 acc: 0.9929 | val loss: 0.0324 acc:
0.9907
[efficientnet] epoch 30/50 | train loss: 0.0178 acc: 0.9931 | val loss: 0.0372 acc:
0.9895
[efficientnet] epoch 31/50 | train loss: 0.0167 acc: 0.9947 | val loss: 0.0364 acc:
0.9895
[efficientnet] epoch 32/50 | train loss: 0.0145 acc: 0.9948 | val loss: 0.0350 acc:
0.9854
early stopping triggered at epoch 33
efficientnet done!
```

Evalutation

In [17]:

```
"""
Model evaluation function for the test dataset.
- Disables gradient tracking to speed up inference and save memory.
- Iterates through the test data to collect predictions, true labels, and class pro
- Outputs a detailed classification report and calculates the multi-class ROC-AUC s
"""

def evaluate_model(model, test_loader, model_name):
    model.eval()                                                    # set to evalu
    all_preds, all_labels, all_probs = [], [], []

    with torch.no_grad():                                           # turn off gra
        for images, labels in test_loader:
            images = images.to(device)
```

```
            labels = labels.squeeze().long()
            outputs = model(images)                                # forward pass

            # apply softmax to get probabilities (needed for ROC-AUC)
            probs = torch.softmax(outputs, dim=1).cpu().numpy()
            _, predicted = outputs.max(1)                          # get final cl
            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(labels.numpy())
            all_probs.extend(probs)

    # convert lists to numpy arrays
    all_preds  = np.array(all_preds)
    all_labels = np.array(all_labels)
    all_probs  = np.array(all_probs)

    # classification report
    print(f"\n=== {model_name} test results ===")
    print(classification_report(all_labels, all_preds, digits=4))

    auc = roc_auc_score(all_labels, all_probs, multi_class='ovr')
    print(f"ROC-AUC (macro ovr): {auc:.4f}")

    return all_preds, all_labels, all_probs
```

In [18]:

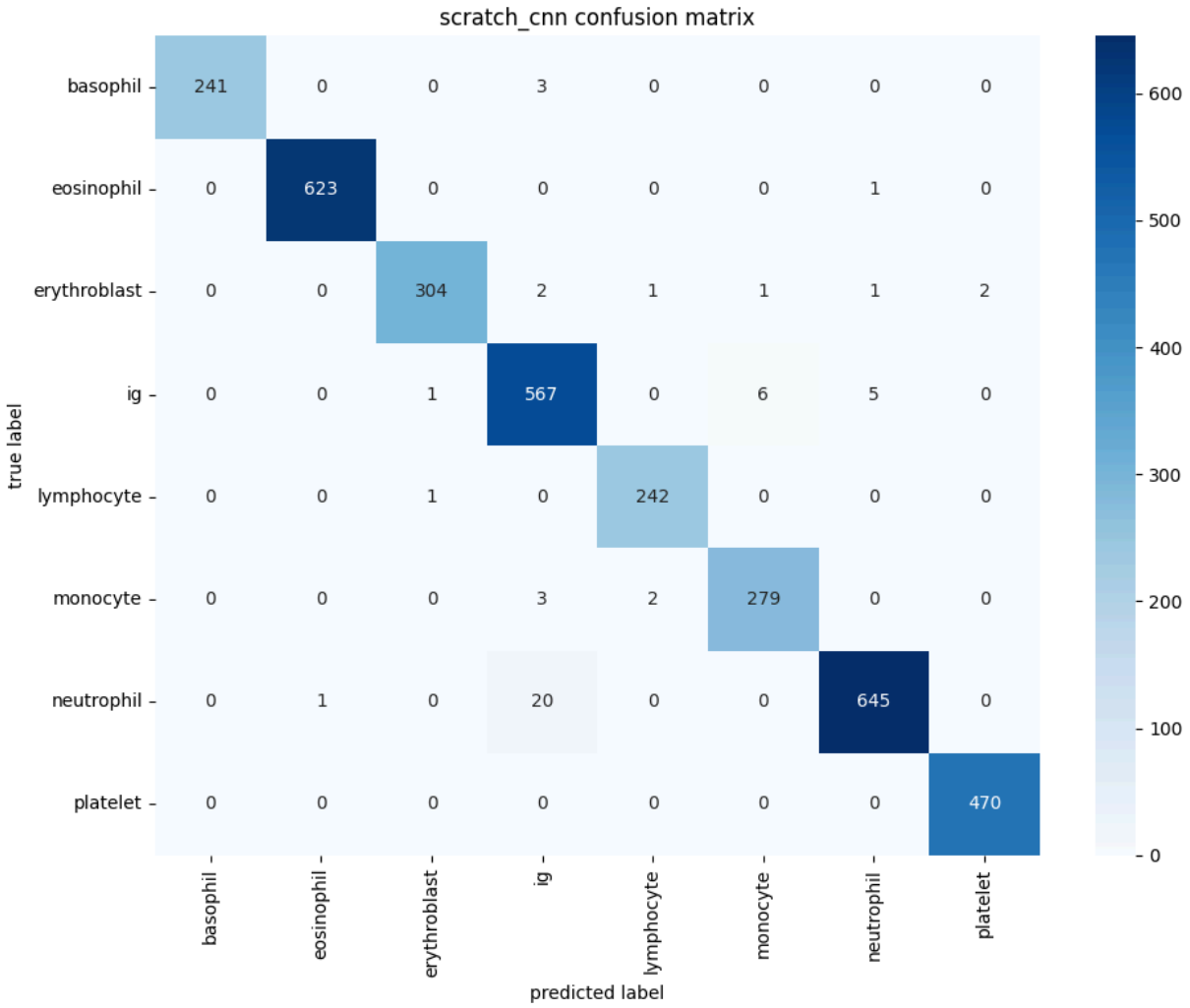
```
# evaluate all 3 models with classification report and ROC/AUC scores
cnn_preds, cnn_labels, cnn_probs           = evaluate_model(model, test_loader, "sc
resnet_preds, resnet_labels, resnet_probs  = evaluate_model(resnet_model, test_load
eff_preds, eff_labels, eff_probs           = evaluate_model(efficientnet_model, tes
```

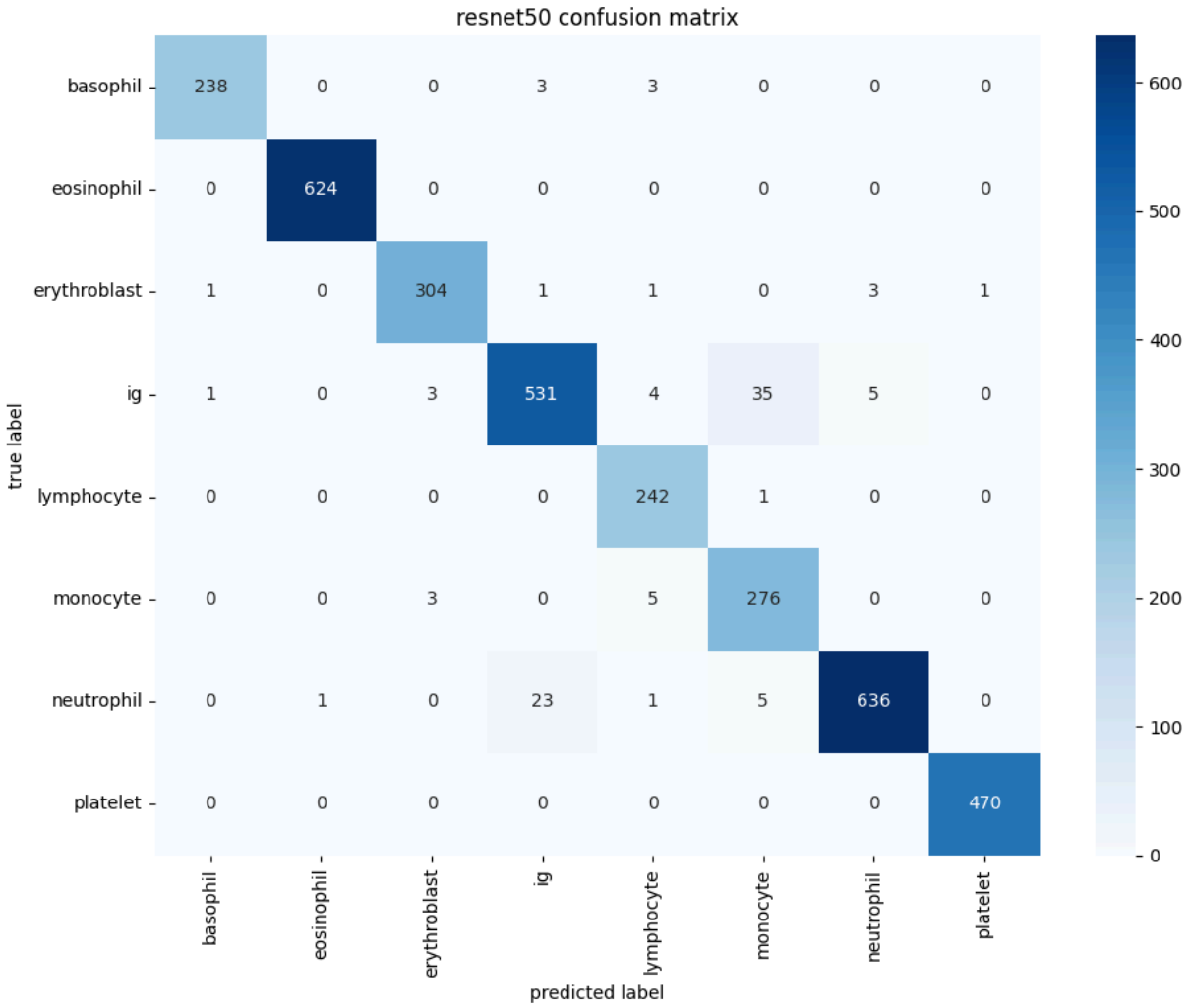
```
=== scratch CNN test results ===
              precision    recall  f1-score   support

           0     1.0000    0.9877    0.9938       244
           1     0.9984    0.9984    0.9984       624
           2     0.9935    0.9775    0.9854       311
           3     0.9529    0.9793    0.9659       579
           4     0.9878    0.9959    0.9918       243
           5     0.9755    0.9824    0.9789       284
           6     0.9893    0.9685    0.9788       666
           7     0.9958    1.0000    0.9979       470

    accuracy                         0.9854      3421
   macro avg     0.9866    0.9862    0.9864      3421
weighted avg     0.9856    0.9854    0.9854      3421

ROC-AUC (macro ovr): 0.9993

=== resnet50 test results ===
              precision    recall  f1-score   support

           0     0.9917    0.9754    0.9835       244
           1     0.9984    1.0000    0.9992       624
           2     0.9806    0.9775    0.9791       311
           3     0.9516    0.9171    0.9340       579
           4     0.9453    0.9959    0.9699       243
           5     0.8707    0.9718    0.9185       284
           6     0.9876    0.9550    0.9710       666
           7     0.9979    1.0000    0.9989       470

    accuracy                         0.9708      3421
   macro avg     0.9655    0.9741    0.9693      3421
weighted avg     0.9718    0.9708    0.9709      3421

ROC-AUC (macro ovr): 0.9986

=== efficientnet test results ===
              precision    recall  f1-score   support

           0     0.9918    0.9959    0.9939       244
           1     0.9984    1.0000    0.9992       624
           2     0.9967    0.9807    0.9887       311
           3     0.9707    0.9741    0.9724       579
           4     0.9758    0.9959    0.9857       243
           5     0.9893    0.9754    0.9823       284
           6     0.9789    0.9760    0.9774       666
           7     0.9958    1.0000    0.9979       470

    accuracy                         0.9866      3421
   macro avg     0.9872    0.9872    0.9872      3421
weighted avg     0.9866    0.9866    0.9865      3421

ROC-AUC (macro ovr): 0.9994
```

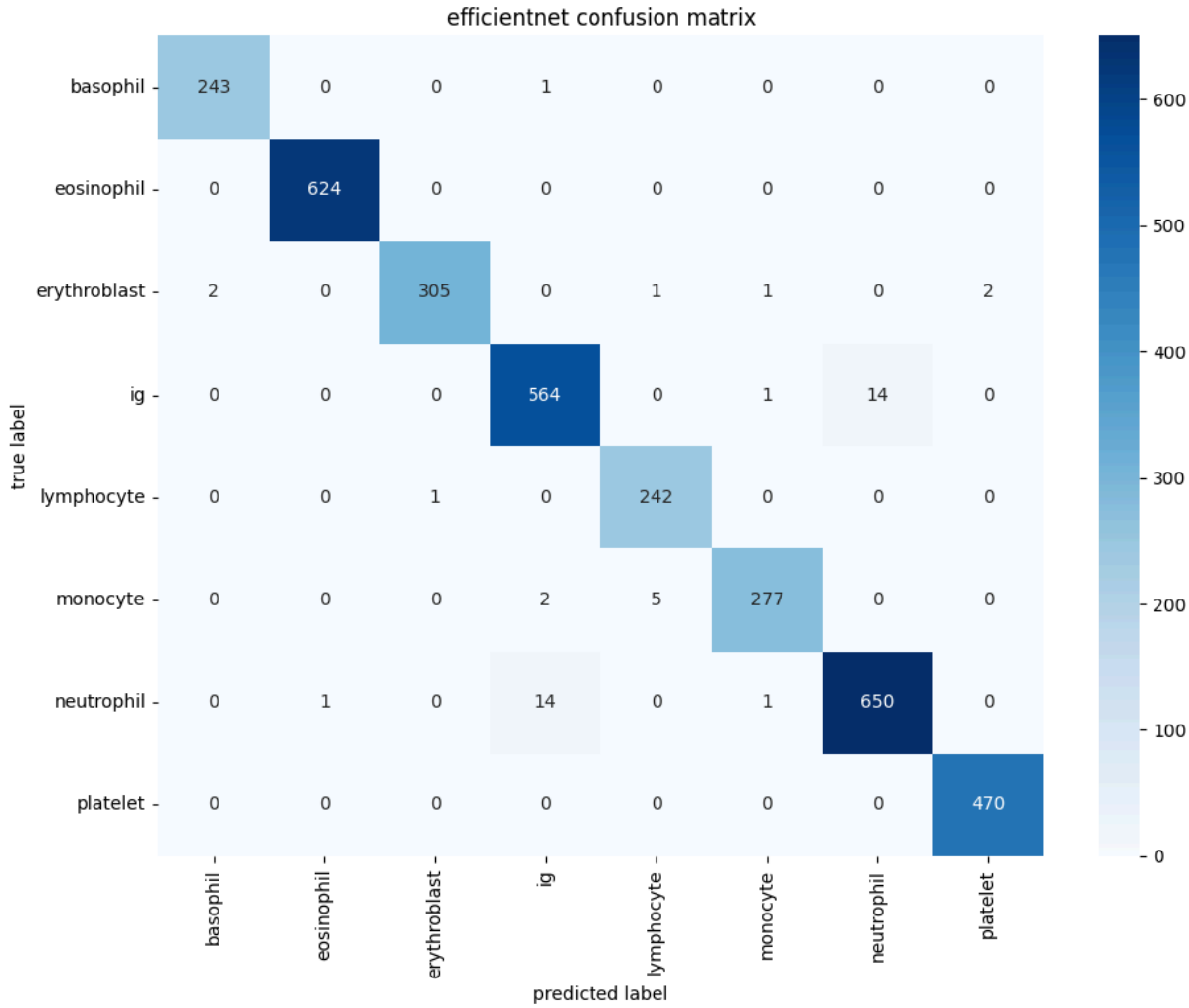
In [27]:

```
"""
Confusion matrix visualization for model comparison.
```

```
- Maps numerical predictions back to their specific blood cell string labels.
- Generates a color-coded heatmap (via seaborn) to easily spot which classes the mo
- Saves each generated matrix to the designated figures folder.
"""

# NOTE: ig = Immature granulocytes (myelocytes, metamyelocytes, and promyelocytes)

cell_types = ['basophil', 'eosinophil', 'erythroblast', 'ig', 'lymphocyte', 'monocy

# define a function to create the confusion matrix for said model
def plot_confusion_matrix(labels, preds, model_name):

    # calculate the raw counts of true vs predicted classifications
    cm = confusion_matrix(labels, preds)
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=cell_types, yticklabels=cell_types)
    plt.title(f'{model_name} confusion matrix')
    plt.ylabel('true label')
    plt.xlabel('predicted label')
    plt.tight_layout()
    # save each plot as a png
    plt.savefig(os.path.join(figures_folder, f'confusion_matrix_{model_name}.png'),
    plt.show()

# create the confusion matrix for each of the 3 plots
plot_confusion_matrix(cnn_labels, cnn_preds, 'scratch_cnn')
plot_confusion_matrix(resnet_labels, resnet_preds, 'resnet50')
plot_confusion_matrix(eff_labels, eff_preds, 'efficientnet')
```


scratch_cnn confusion matrix
true label
predicted label
basophil
eosinophil
erythroblast
ig
lymphocyte
monocyte
neutrophil
platelet
241 0 0 3 0 0 0 0
0 623 0 0 0 0 1 0
0 0 304 2 1 1 1 2
0 0 1 567 0 6 5 0
0 0 1 0 242 0 0 0
0 0 0 3 2 279 0 0
0 1 0 20 0 0 645 0
0 0 0 0 0 0 0 470
600
500
400
300
200
100
0

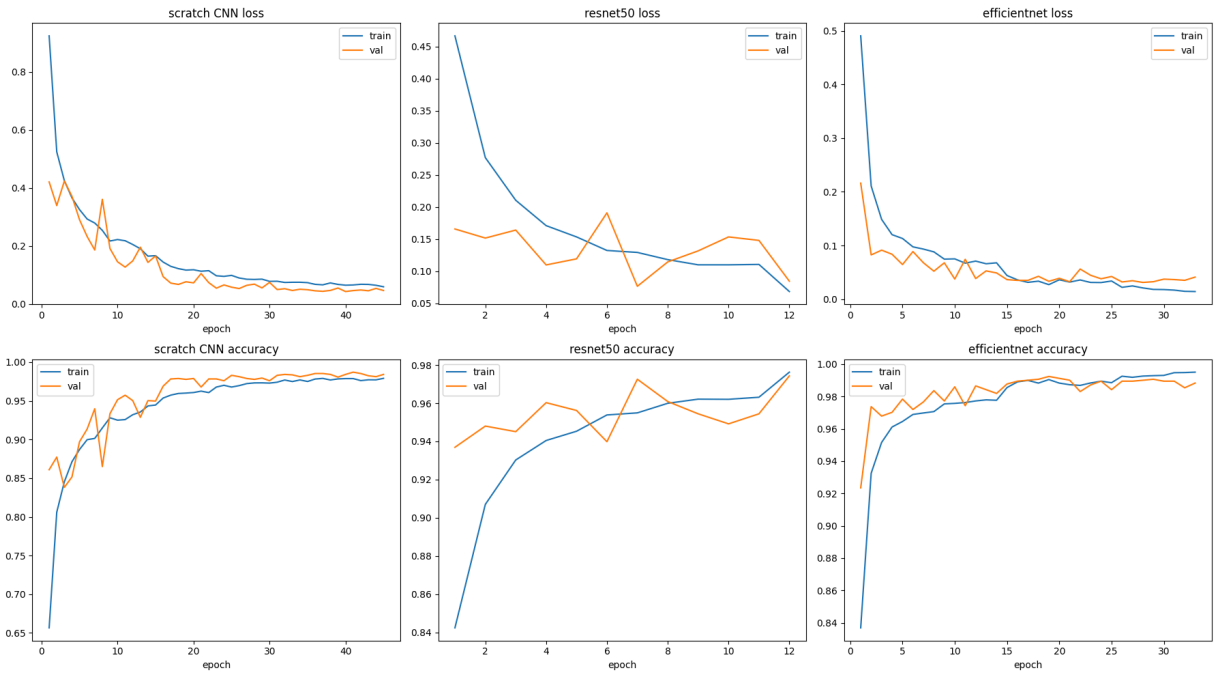

resnet50 confusion matrix
true label
predicted label
basophil
eosinophil
erythroblast
ig
lymphocyte
monocyte
neutrophil
platelet
238 0 0 3 3 0 0 0
0 624 0 0 0 0 0 0
1 0 304 1 1 0 3 1
1 0 3 531 4 35 5 0
0 0 0 0 242 1 0 0
0 0 3 0 5 276 0 0
0 1 0 23 1 5 636 0
0 0 0 0 0 0 0 470
600
500
400
300
200
100
0



In [29]:

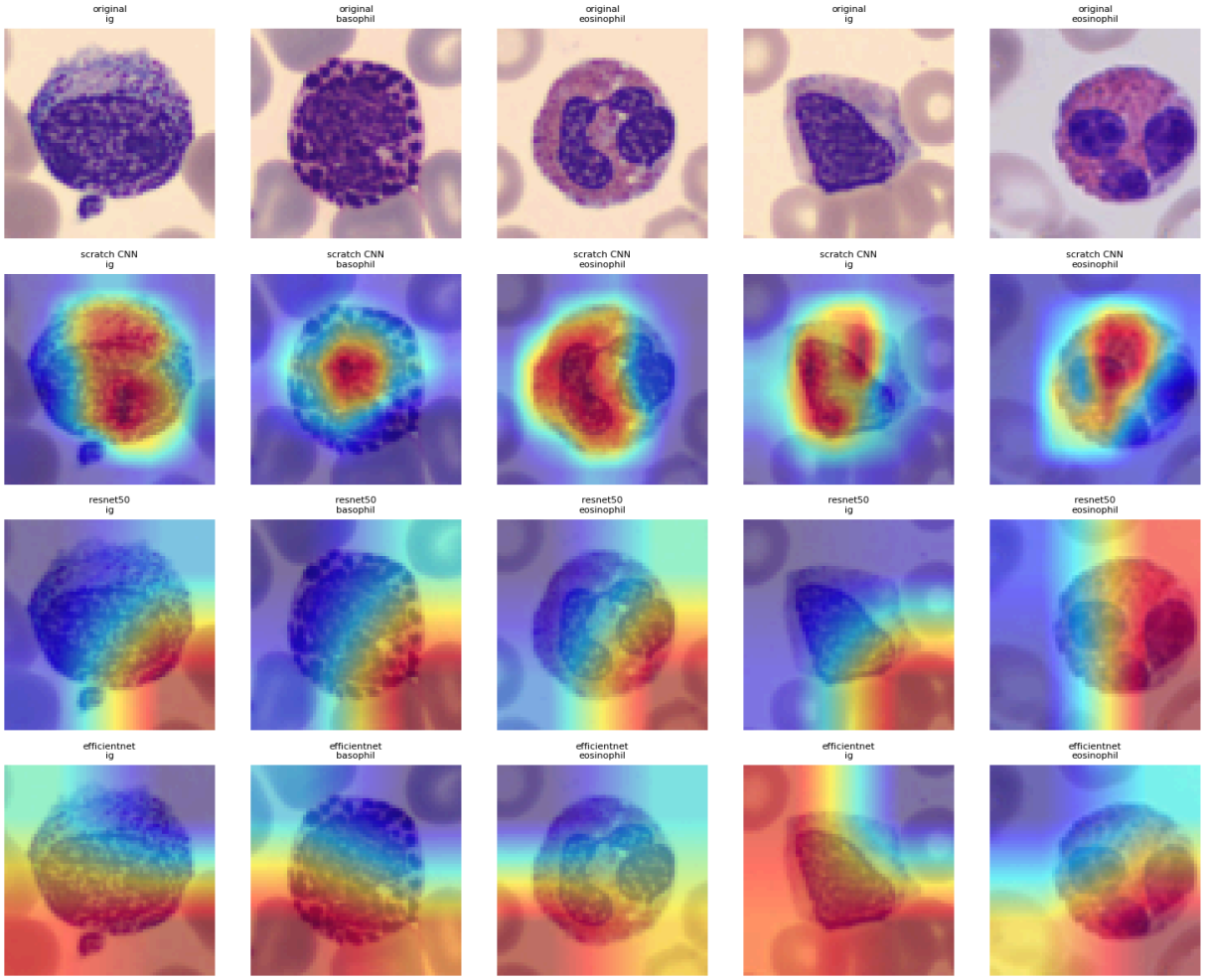
```
"""
Comprehensive training curve visualization.
- Aggregates the results from the Custom CNN, ResNet50, and EfficientNet.
- Generates a 2x3 subplot grid comparing the loss (top row) and accuracy (bottom ro
- Saves the final comparative figure to the designated figures folder.
"""

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# # bundle all results into a dictionary for clean iteration
results = {
    'scratch CNN': cnn_results,
    'resnet50': resnet_results,
    'efficientnet': efficientnet_results
}

for i, (name, (t_loss, v_loss, t_acc, v_acc)) in enumerate(results.items()):
    epochs = range(1, len(t_loss) + 1)

    # top row: loss
    axes[0, i].plot(epochs, t_loss, label='train')
    axes[0, i].plot(epochs, v_loss, label='val')
    axes[0, i].set_title(f'{name} loss')
    axes[0, i].set_xlabel('epoch')
```

```
    axes[0, i].legend()

    # bottom row: accuracy
    axes[1, i].plot(epochs, t_acc, label='train')
    axes[1, i].plot(epochs, v_acc, label='val')
    axes[1, i].set_title(f'{name} accuracy')
    axes[1, i].set_xlabel('epoch')
    axes[1, i].legend()

plt.tight_layout()
plt.savefig(os.path.join(figures_folder, f'all_models_training_curves.png'), dpi=15
plt.show()
```



Grad CAM Plots

In [30]:

```
"""
Grad-CAM (Gradient-weighted Class Activation Mapping) visualizations.
- Initializes Grad-CAM for all three models by targeting their final convolutional
- Un-normalizes a sample of test images to display them in standard RGB format.
- Generates a 4x5 grid comparing the original images against the heatmaps produced
"""

# pick target layer for each model
cam_cnn = GradCAM(model=model, target_layers=[model.features[-2]])
cam_resnet = GradCAM(model=resnet_model, target_layers=[resnet_model.layer4[-1]])
cam_efficient = GradCAM(model=efficientnet_model, target_layers=[efficientnet_model

# grab a few test images
sample_images, sample_labels = next(iter(test_loader))
sample_images = sample_images[:5].to(device)

fig, axes = plt.subplots(4, 5, figsize=(15, 12))
```

```
# row 0: original images
for j in range(5):
    img = sample_images[j].cpu().permute(1, 2, 0).numpy()
    img = img * [0.229, 0.224, 0.225] + [0.485, 0.456, 0.406]
    img = np.clip(img, 0, 1)
    axes[0, j].imshow(img)
    axes[0, j].set_title(f'original\n{cell_types[sample_labels[j].item()]}', fontsi
    axes[0, j].axis('off')

# rows 1-3: gradcam for each model
for i, (cam, name) in enumerate([(cam_cnn, 'scratch CNN'), (cam_resnet, 'resnet50')
    grayscale_cams = cam(input_tensor=sample_images)
    for j in range(5):
        img = sample_images[j].cpu().permute(1, 2, 0).numpy()
        img = img * [0.229, 0.224, 0.225] + [0.485, 0.456, 0.406]
        img = np.clip(img, 0, 1).astype(np.float32)
        cam_image = show_cam_on_image(img, grayscale_cams[j], use_rgb=True)
        axes[i+1, j].imshow(cam_image)
        axes[i+1, j].set_title(f'{name}\n{cell_types[sample_labels[j].item()]}', fo
        axes[i+1, j].axis('off')

plt.tight_layout()
plt.savefig(os.path.join(figures_folder, f'gradcam.png'), dpi=150)
plt.show()
```